FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

Faculté des Sciences Ben M'Sick - CASABLANCA

Rapport de Projet Cloud

Infrastructure CI/CD avec Kubernetes Terraform

Étudiante :
Marwa Bouzaouit
Aya Adadi
Malak Boussetta
Zakaria Gasmi

enseignante:
Prof. Benabbou Fouzia

Année Universitaire 2025-2026

Abstract

In today's fast-evolving technological environment, ensuring rapid deployment, scalability, and system reliability has become a major challenge. DevOps practices have emerged as an effective solution to bridge the gap between development and operations while automating the entire application lifecycle.

This project focuses on the design and implementation of a complete DevOps infrastructure based on automation, containerization, and continuous deployment. The infrastructure provisioning is achieved using **Terraform**, enabling the creation of cloud resources in a reproducible and efficient way. Configuration management and cluster setup are automated using **Ansible**, ensuring consistent and reliable environments.

At the core of this project, a **Python Flask** application is developed and containerized using **Docker**, allowing portability across different environments. The application is then deployed on a **Kubernetes** cluster, ensuring scalability and efficient orchestration of containers.

Finally, a **CI/CD pipeline** is implemented to automate the process of testing, building, and deploying the application. This pipeline ensures continuous integration and delivery, reducing manual intervention and improving deployment speed.

This project demonstrates how a complete DevOps approach can simplify infrastructure management, enhance system reliability, and accelerate application delivery.

Keywords : DevOps, Terraform, Ansible, Docker, Kubernetes, CI/CD, Cloud Computing, Flask

Resumé

Dans un environnement technologique en constante évolution, la rapidité de déploiement, la scalabilité et la fiabilité des systèmes sont devenues des enjeux majeurs. Les pratiques DevOps apportent une solution efficace en permettant d'automatiser le cycle de vie des applications tout en améliorant la collaboration entre le développement et l'exploitation.

Ce projet a pour objectif la conception et la mise en place d'une infrastructure DevOps complète basée sur l'automatisation, la conteneurisation et le déploiement continu. Le provisionnement de l'infrastructure est réalisé à l'aide de **Terraform**, permettant de créer des ressources cloud de manière reproductible et efficace. La configuration des machines et l'installation du cluster sont automatisées grâce à **Ansible**, garantissant un environnement cohérent et fiable.

Au cœur du projet, une application développée en **Python** avec le framework **Flask** est conteneurisée à l'aide de **Docker**, assurant sa portabilité entre différents environnements. Cette application est ensuite déployée sur un cluster **Kubernetes**, permettant une gestion efficace et une scalabilité des services.

Enfin, un pipeline **CI/CD** est mis en place afin d'automatiser les étapes de test, de build et de déploiement de l'application. Cette approche réduit les interventions manuelles et améliore la rapidité ainsi que la fiabilité des mises en production.

Ce projet démontre l'importance d'une approche DevOps complète pour simplifier la gestion de l'infrastructure, améliorer la fiabilité des systèmes et accélérer le déploiement des applications.

Mots-clés : DevOps, Terraform, Ansible, Docker, Kubernetes, CI/CD, Cloud Computing, Flask

Table des matières

Abstract **1**

Resumé **2**

Introduction **5**

1 Architecture Globale du Projet **6**
1.1 Introduction 6
1.2 Description des Composants 6
1.3 Flux Global du Système 7
1.4 Tableau Récapitulatif des Outils 8
1.5 Conclusion 8

2 Mise en Place de l'Infrastructure avec Terraform **9**
2.1 Introduction à l'Infrastructure as Code (IaC) 9
2.2 Structure du Projet Terraform 9
2.3 Configuration du fichier `main.tf` 10
2.3.1 Bloc Provider 10
2.3.2 Ressource aws_instance "master" 11
2.3.3 Ressource aws_instance "worker" 11
2.4 Déploiement de l'Infrastructure 11
2.4.1 Initialisation du projet 11
2.4.2 Planification 11
2.4.3 Application du plan 13
2.5 Résultat : Instances EC2 opérationnelles 14

3 Configuration et Installation du Cluster Kubernetes avec Ansible **15**
3.1 Introduction à Ansible 15
3.2 Structure du Projet Ansible 15
3.3 Fichier d'Inventaire : inventory.ini 16
3.3.1 Groupes de l'inventaire 16
3.4 Fichier de Playbook : playbook.yml 16
3.4.1 Tâches du Playbook 17
3.5 Test de Connectivité avec le Module Ping 18
3.6 Exécution du Playbook 18
3.6.1 Phases de l'exécution 19
3.6.2 Récapitulatif final 19
3.7 Récapitulatif des Tâches Ansible 19
3.8 Conclusion 19

4 Développement et Conteneurisation de l'Application **20**
4.1 Présentation de l'application 20
4.2 Conteneurisation avec Docker 20
4.2.1 Structure du Dockerfile 21
4.2.2 Construction de l'image 21
4.3 Publication de l'image sur Docker Hub 21
4.4 Déploiement sur Kubernetes (Orchestration) 22
4.4.1 Deployment (deployment.yaml) 22
4.4.2 Service (service.yaml) 22
4.4.3 Validation du déploiement 23

5 Pipeline CI/CD avec GitHub Actions **24**
5.1 Introduction au CI/CD 24
5.2 Structure du pipeline 24
5.3 Étapes du pipeline (Workflow Jobs) 24
5.4 Sécurité et gestion des Secrets 24
5.5 Résultat : Pipeline en action 25
5.6 Test de validation End-to-End 26

Conclusion **27**

Introduction

Dans un contexte où les systèmes informatiques deviennent de plus en plus complexes et exigeants, les entreprises doivent faire face à des défis majeurs tels que la rapidité de déploiement, la fiabilité des applications et la gestion efficace des infrastructures. Dans ce cadre, l'approche **DevOps** s'impose comme une solution incontournable, permettant d'améliorer la collaboration entre les équipes de développement et d'exploitation tout en automatisant les processus de livraison logicielle.

L'objectif de ce projet est de concevoir et de mettre en œuvre une infrastructure DevOps complète, allant du développement de l'application jusqu'à son déploiement automatique sur un environnement cloud. Cette approche repose sur l'utilisation de plusieurs outils modernes permettant d'automatiser chaque étape du cycle de vie de l'application.

Dans un premier temps, l'infrastructure cloud est provisionnée à l'aide de **Terraform**, en adoptant le concept d'*Infrastructure as Code (IaC)*, ce qui permet de garantir la reproductibilité et la cohérence des ressources créées. Ensuite, la configuration des machines et l'installation du cluster **Kubernetes** sont automatisées grâce à **Ansible**.

Par la suite, une application web développée avec le framework **Flask** est conteneurisée à l'aide de **Docker** afin d'assurer sa portabilité et sa facilité de déploiement. Cette application est ensuite déployée sur un cluster **Kubernetes**, permettant une orchestration efficace des conteneurs et une meilleure gestion de la scalabilité.

Enfin, un pipeline **CI/CD** est mis en place pour automatiser les étapes de test, de construction et de déploiement de l'application. Cette chaîne d'intégration et de livraison continue permet de réduire les erreurs humaines, d'accélérer les mises en production et d'assurer une meilleure qualité du logiciel.

Ce projet constitue ainsi une mise en pratique concrète des concepts DevOps, en illustrant la construction d'une chaîne complète et automatisée, depuis le code source jusqu'au déploiement final sur une infrastructure cloud.

Chapitre 1

Architecture Globale du Projet

1.1 Introduction

Dans le cadre de ce projet, nous avons conçu et déployé une infrastructure **DevOps** complète, intégrant l'ensemble des phases du cycle de vie d'une application logicielle : du développement local jusqu'à la mise en production automatisée sur le cloud.

L'objectif principal était de construire un pipeline entièrement automatisé, capable de transformer chaque modification du code source en une version déployée et accessible de l'application, sans aucune intervention manuelle.

Cette approche repose sur l'intégration de plusieurs outils complémentaires, chacun jouant un rôle précis dans la chaîne de livraison logicielle. Ensemble, ils forment ce que l'on appelle un pipeline **DevOps de bout en bout**.

1.2 Description des Composants

L'infrastructure mise en place repose sur plusieurs composants principaux qui interagissent de manière séquentielle et automatisée.

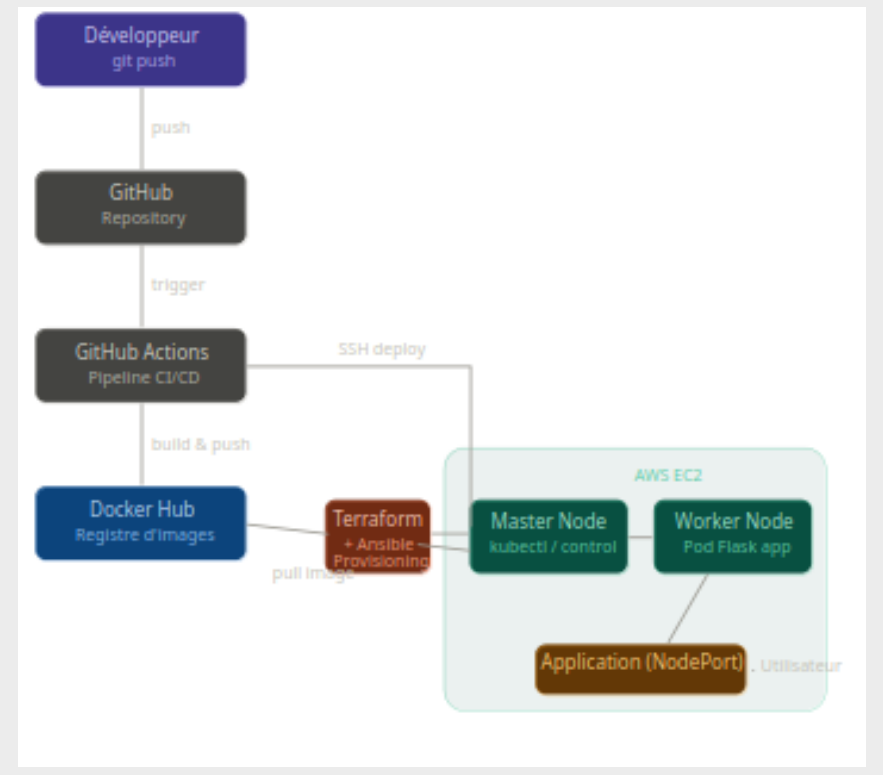
Le développeur et le dépôt **GitHub** constituent le point de départ du pipeline. Chaque modification du code source est soumise via une commande `git push` vers le dépôt distant. Cette action déclenche automatiquement l'ensemble du processus de build et de déploiement.

GitHub Actions représente le moteur du pipeline **CI/CD**. Dès qu'un *push* est détecté sur la branche principale, un workflow est lancé automatiquement. Ce workflow orchestre plusieurs étapes : construction de l'image Docker, publication sur **Docker Hub**, connexion SSH aux serveurs et mise à jour du déploiement Kubernetes.

Docker Hub joue le rôle de registre centralisé d'images. L'image de l'application **Flask** est construite puis stockée sur cette plateforme, permettant aux nœuds du cluster Kubernetes de la récupérer lors du déploiement.

Terraform et **Ansible** assurent respectivement le provisionnement de l'infrastructure cloud et la configuration des machines. Terraform permet de créer les instances **EC2** sur **AWS**, tandis qu'Ansible installe et configure les outils nécessaires comme Docker et Kubernetes.

Le cluster **Kubernetes**, composé d'un nœud **Master** et d'un nœud **Worker**, assure l'orchestration des conteneurs. Le nœud Master gère le plan de contrôle, tandis que le nœud Worker exécute les pods contenant l'application. L'application est exposée via un service de type **NodePort**.



1.3 Flux Global du Système

Le fonctionnement global du système suit les étapes suivantes :

1. Le développeur effectue un `git push` vers le dépôt GitHub.
2. GitHub Actions détecte le push et déclenche le pipeline CI/CD.
3. Le pipeline construit l'image Docker et la publie sur Docker Hub.
4. Le pipeline se connecte en SSH au nœud Master (EC2).
5. Kubernetes télécharge l'image et met à jour les pods sur le nœud Worker.
6. L'application devient accessible via le navigateur grâce au service NodePort.

1.4 Tableau Récapitulatif des Outils

Outil	Rôle	Description
GitHub	Gestion du code	Hébergement du code source et déclenchement du pipeline CI/CD
GitHub Actions	CI/CD	Automatisation des étapes de build, test et déploiement
Docker	Conteneurisation	Création d'images portables de l'application
Docker Hub	Registre	Stockage et distribution des images Docker
Terraform	Infrastructure	Provisionnement automatique des ressources cloud (EC2, VPC)
Ansible	Configuration	Installation et configuration des outils sur les machines
Kubernetes	Orchestration	Gestion des conteneurs, déploiement et scalabilité
AWS	Cloud	Fourniture des ressources (machines virtuelles, réseau)

Table 1.1 – Tableau récapitulatif des outils utilisés

1.5 Conclusion

L'architecture proposée met en œuvre une chaîne DevOps complète et automatisée, permettant de passer du développement de l'application à son déploiement sur le cloud sans intervention manuelle.

Elle offre plusieurs avantages :

— une meilleure fiabilité
— une rapidité de déploiement
— une infrastructure reproductible
— une gestion efficace des applications

Ce chapitre constitue une base essentielle pour la compréhension du projet et introduit les technologies qui seront détaillées dans les chapitres suivants.

Chapitre 2

Mise en Place de l'Infrastructure avec Terraform

2.1 Introduction à l'Infrastructure as Code (IaC)

La gestion manuelle des infrastructures informatiques présente de nombreux inconvénients : elle est sujette aux erreurs humaines, difficile à reproduire, et ne s'adapte pas facilement à des environnements complexes. C'est dans ce contexte qu'est née la notion d'**Infrastructure as Code (IaC)**, une approche qui consiste à définir et gérer l'infrastructure informatique à travers des fichiers de configuration déclaratifs, de la même manière que l'on écrit du code source.

L'IaC permet ainsi de versionner l'infrastructure, de la reproduire de manière identique dans différents environnements, et d'automatiser entièrement son déploiement. Cette approche est aujourd'hui un pilier fondamental des pratiques DevOps modernes.

Dans le cadre de ce projet, nous avons adopté **Terraform**, l'outil IaC développé par **HashiCorp**, pour provisionner notre infrastructure cloud sur **AWS**. Terraform utilise un langage déclaratif appelé **HCL (HashiCorp Configuration Language)** qui permet de décrire les ressources à créer, modifier ou supprimer, puis de les appliquer de façon automatisée et reproductible.

2.2 Structure du Projet Terraform

Le projet Terraform est organisé dans un répertoire dédié contenant les fichiers de configuration nécessaires à la création de l'infrastructure.

Figure 2.1 — Structure du projet Terraform dans VS Code

```
main.tf
terraform.tfstate
terraform.tfstate.backup
.terraform/
terraform.lock.hcl
```

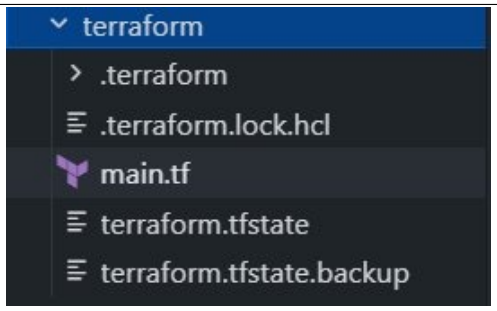
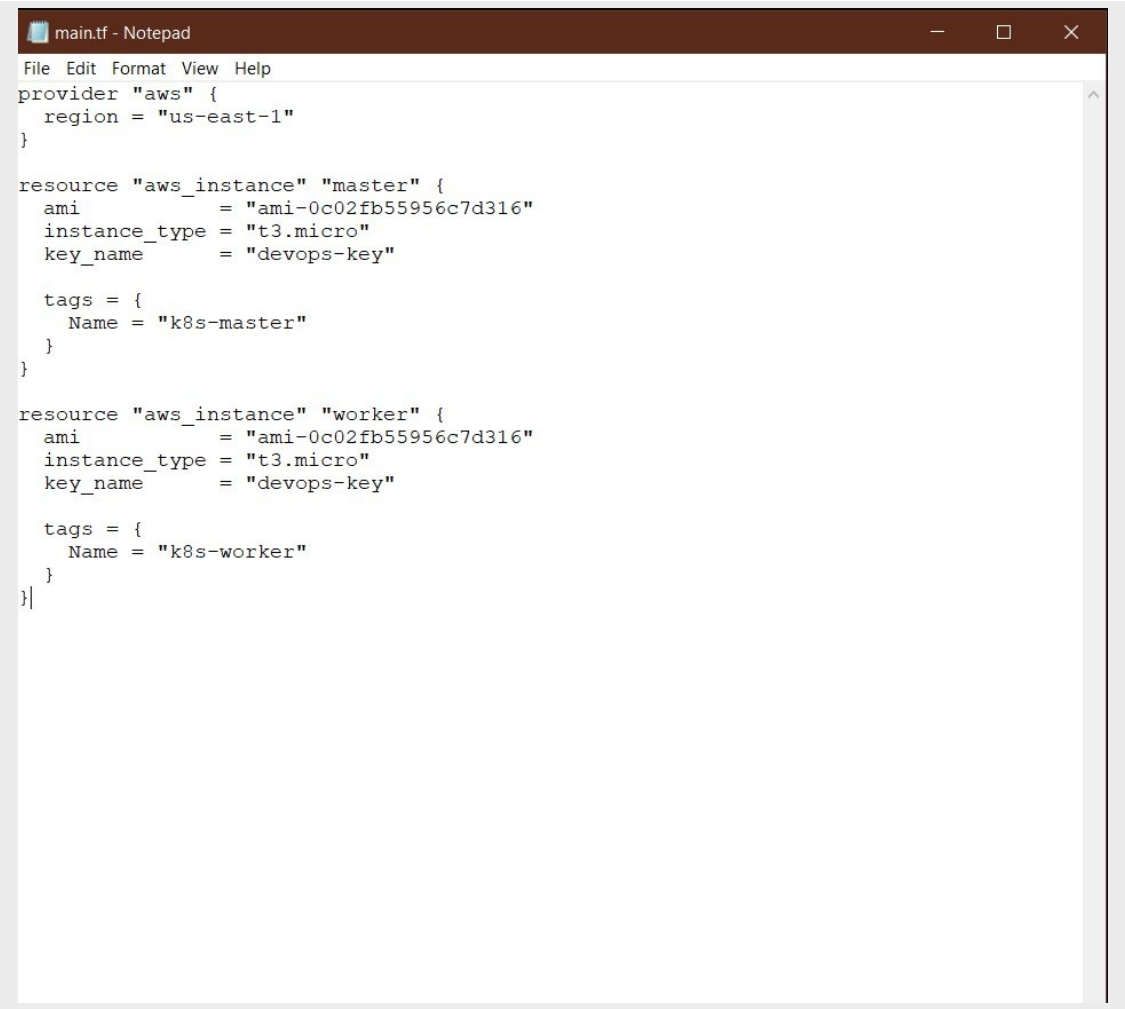


FIGURE 2.1 – Structure des fichiers Terraform

— `main.tf` : fichier principal contenant la définition des ressources AWS à provisionner (instances EC2, provider, etc.).
— `terraform.tfstate` : fichier d'état généré automatiquement par Terraform après chaque exécution.
— `terraform.tfstate.backup` : sauvegarde automatique de l'état précédent.
— `.terraform/` : répertoire contenant les plugins et providers téléchargés.
— `terraform.lock.hcl` : fichier de verrouillage des versions des providers utilisés.

2.3 Configuration du fichier `main.tf`

Le fichier `main.tf` constitue le cœur de la configuration Terraform. Il contient le **provider AWS** ainsi que les deux ressources EC2 à créer : le nœud **Master** et le nœud **Worker** du cluster Kubernetes.



2.3.1 Bloc Provider

```
provider "aws" {
  region = "us-east-1"
}
```

Le bloc provider "aws" définit la région AWS dans laquelle les ressources seront créées. Nous avons choisi la région us-east-1 (Virginie du Nord), qui offre une large disponibilité des types d'instances et une faible latence pour nos besoins.

2.3.2 Ressource aws_instance "master"

Les paramètres configurés sont les suivants :

- `ami` : identifiant de l'AMI (**Amazon Machine Image**) Ubuntu utilisée comme système d'exploitation de base (`ami-0c02fb55956c7d316`).
- `instance_type` : type d'instance choisi, ici `t3.micro`, adapté à un environnement de développement et de test.
- `key_name` : nom de la paire de clés SSH (`devops-key`) permettant l'accès sécurisé à la machine.
- `tags` : balise `Name` permettant d'identifier facilement l'instance dans la console AWS sous le nom `k8s-master`.

2.3.3 Ressource aws_instance "worker"

Ce bloc est identique dans sa structure au bloc **Master**, à l'exception du tag `Name` qui est défini comme `k8s-worker`. Les deux instances partagent le même `instance_type` et la même `AMI`, assurant ainsi une cohérence dans la configuration du cluster **Kubernetes**.

2.4 Déploiement de l'Infrastructure

Le déploiement de l'infrastructure avec Terraform se déroule en trois étapes successives, chacune correspondant à une commande spécifique.

2.4.1 Initialisation du projet

```
$ terraform init
```

Cette commande télécharge et installe les plugins nécessaires, notamment le provider AWS.

2.4.2 Planification

```
$ terraform plan
```

Cette commande génère un plan d'exécution détaillant les actions que Terraform effectuera.

Figure 2.3 — Plan Terraform pour l'instance k8s-master

```
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.master will be created
  + resource "aws_instance" "master" {
      + ami                                  = "ami-0c02fb55956c7d316"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "devops-key"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + region                               = "us-east-1"
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
      + subnet_id                            = (known after apply)
      + tags                                 = {
          + "Name" = "k8s-master"
        }
      + tags_all                             = {
          + "Name" = "k8s-master"
        }
      + tenancy                              = (known after apply)
```

Figure 2.4 — Plan Terraform pour l'instance k8s-worker

```
# aws_instance.worker will be created
+ resource "aws_instance" "worker" {
    + ami                                  = "ami-0c02fb55956c7d316"
    + arn                                  = (known after apply)
    + associate_public_ip_address          = (known after apply)
    + availability_zone                    = (known after apply)
    + disable_api_stop                     = (known after apply)
    + disable_api_termination              = (known after apply)
    + ebs_optimized                        = (known after apply)
    + enable_primary_ipv6                  = (known after apply)
    + force_destroy                        = false
    + get_password_data                    = false
    + host_id                              = (known after apply)
    + host_resource_group_arn              = (known after apply)
    + iam_instance_profile                 = (known after apply)
    + id                                   = (known after apply)
    + instance_initiated_shutdown_behavior = (known after apply)
    + instance_lifecycle                   = (known after apply)
    + instance_state                       = (known after apply)
    + instance_type                        = "t3.micro"
    + ipv6_address_count                   = (known after apply)
    + ipv6_addresses                       = (known after apply)
    + key_name                             = "devops-key"
    + monitoring                           = (known after apply)
    + outpost_arn                          = (known after apply)
    + password_data                        = (known after apply)
    + placement_group                      = (known after apply)
    + placement_group_id                   = (known after apply)
    + placement_partition_number           = (known after apply)
    + primary_network_interface_id         = (known after apply)
    + private_dns                          = (known after apply)
    + private_ip                           = (known after apply)
    + public_dns                           = (known after apply)
    + public_ip                            = (known after apply)
    + region                               = "us-east-1"
    + secondary_private_ips                = (known after apply)
    + security_groups                      = (known after apply)
    + source_dest_check                    = true
    + spot_instance_request_id             = (known after apply)
    + subnet_id                            = (known after apply)
    + tags                                 = {
        + "Name" = "k8s-worker"
      }
    + tags_all                             = {
        + "Name" = "k8s-worker"
      }
    + tenancy                              = (known after apply)
    + user_data_base64                     = (known after apply)
    + user_data_replace_on_change          = false
    + vpc_security_group_ids               = (known after apply)
```

Le signe « + » en début de chaque ligne indique qu'il s'agit d'une ressource à créer. Les valeurs marquées `(known after apply)` seront déterminées par AWS au moment de la création (identifiant de l'instance, adresse IP publique, etc.).

Les paramètres déjà connus, tels que `instance_type = "t3.micro"`, `ami`, `key_name` et les `tags`, sont affichés directement.

2.4.3 Application du plan

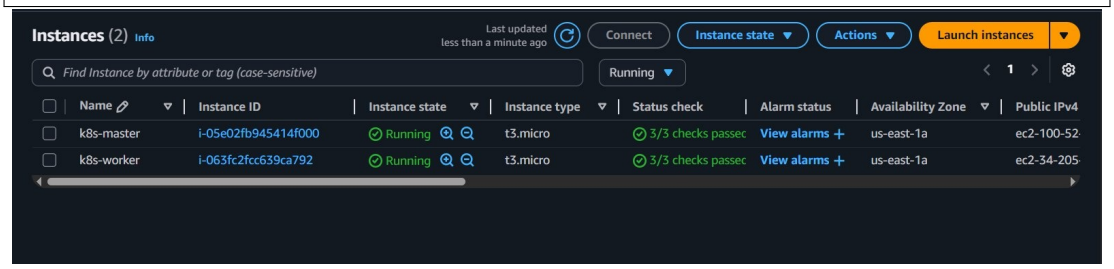
```
$ terraform apply
```

Après validation du plan, la commande terraform apply est exécutée pour créer effectivement les ressources sur AWS. Terraform affiche le résultat de chaque opération et met à jour le fichier terraform.tfstate avec l'état réel de l'infrastructure.

2.5 Résultat : Instances EC2 opérationnelles

Suite à l'exécution de `terraform apply`, les deux instances sont créées et démarrées sur AWS.

Figure 2.5 — Instances k8s-master et k8s-worker en état Running sur AWS EC2



Paramètre	k8s-master	k8s-worker
Instance ID	i-05e02fb945414f000	i-063fc2fcc639ca792
État	Running	Running
Type	t3.medium	t3.medium
Status checks	3/3 passed	3/3 passed
Zone	us-east-1a	us-east-1a

TABLE 2.1 – Caractéristiques des instances EC2

Les deux instances sont dans la même zone de disponibilité (**us-east-1a**), garantissant une latence minimale entre le nœud Master et le nœud Worker pour les communications internes du cluster Kubernetes.

Chapitre 3

Configuration et Installation du Cluster Kubernetes avec Ansible

3.1 Introduction à Ansible

Une fois l'infrastructure cloud provisionnée avec Terraform, il est nécessaire de configurer les machines créées et d'y installer les logiciels requis pour faire fonctionner le cluster **Kubernetes**. Cette étape aurait pu être réalisée manuellement en se connectant à chaque instance via SSH, mais cette approche serait longue, sujette aux erreurs humaines et difficilement reproductible. C'est pourquoi nous avons opté pour **Ansible**, un outil d'automatisation de la configuration et de la gestion des systèmes.

Ansible est un outil open source développé par Red Hat qui permet d'automatiser les tâches d'administration système, d'installation de logiciels et de configuration de serveurs. Son fonctionnement repose sur des fichiers YAML appelés *playbooks*, qui décrivent de façon déclarative les états souhaités des machines cibles. Un avantage majeur d'Ansible est son architecture **sans agent** (*agentless*) : il communique avec les machines distantes uniquement via SSH, sans nécessiter l'installation d'un logiciel supplémentaire sur les nœuds cibles.

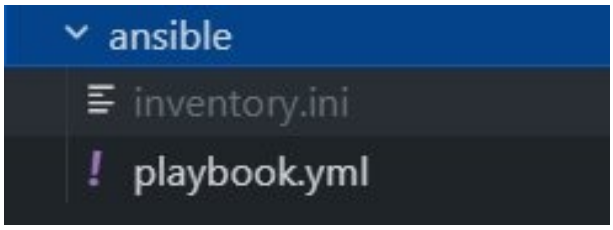
Dans le cadre de ce projet, Ansible a été utilisé pour réaliser deux grandes tâches sur les instances EC2 créées par Terraform :

- Installation et configuration de **Docker**.
- Installation des outils **Kubernetes** (`kubeadm`, `kubelet`, `kubectl`) nécessaires à la mise en place du cluster.

3.2 Structure du Projet Ansible

Le projet Ansible est organisé de manière simple et efficace, avec deux fichiers principaux dans un répertoire dédié nommé `ansible` :

- `inventory.ini` : fichier d'inventaire définissant les machines cibles et les paramètres de connexion SSH.
- `playbook.yml` : fichier de playbook contenant l'ensemble des tâches d'installation et de configuration à exécuter sur les machines.



3.3 Fichier d'Inventaire : inventory.ini

Le fichier `inventory.ini` est le point d'entrée d'Ansible. Il définit les hôtes cibles et les organise en groupes logiques.

```
[master]
98.82.27.94 ansible_user=ec2-user ansible_ssh_private_key_file=/mnt/c/Users/Aya/.ssh/devops-key.pem

[worker]
13.223.187.248 ansible_user=ec2-user ansible_ssh_private_key_file=/mnt/c/Users/Aya/.ssh/devops-key.pem

[all:vars]
ansible_python_interpreter=/usr/bin/python3
```

3.3.1 Groupes de l'inventaire

- **[master]** : adresse IP publique du nœud Master (`98.82.27.94`), avec les paramètres SSH :
  `ansible_user=ec2-user` et `ansible_ssh_private_key_file=devops-key.pem`.
- **[worker]** : adresse IP publique du nœud Worker (`13.223.187.248`), mêmes paramètres SSH que le Master.
- **[all :vars]** : variable globale applicable à tous les hôtes :
  `ansible_python_interpreter=/usr/bin/python3`.

3.4 Fichier de Playbook : playbook.yml

Le fichier `playbook.yml` contient les instructions de configuration à appliquer sur les machines. Il comprend un seul *play* nommé **"Setup Kubernetes nodes"**, ciblant tous les hôtes avec les privilèges administrateur (`become: yes`).

```yaml
- name: Setup Kubernetes nodes
  hosts: all
  become: yes

  tasks:
    - name: Update system packages
      yum:
        name: "*"
        state: latest

    - name: Install Docker
      yum:
        name: docker
        state: present

    - name: Start and enable Docker
      service:
        name: docker
        state: started
        enabled: yes

    - name: Disable swap
      command: swapoff -a

    - name: Install Kubernetes packages
      yum:
        name:
          - kubelet
          - kubeadm
          - kubectl
        state: present

    - name: Enable kubelet
      service:
        name: kubelet
        enabled: yes
```

3.4.1 Tâches du Playbook

1. **Mise à jour des paquets système**

```yaml
- name: Update system packages
  yum:
    name: "*"
    state: latest
```

2. **Installation de Docker**

```yaml
- name: Install Docker
  yum:
    name: docker
    state: present
```

3. **Démarrage et activation de Docker**

```yaml
- name: Start and enable Docker
  service:
    name: docker
    state: started
    enabled: yes
```

4. **Désactivation du swap**

```
- name: Disable swap
  command: swapoff -a
```

5. **Installation des outils Kubernetes**

```
- name: Install Kubernetes packages
  yum:
    name:
      - kubelet
      - kubeadm
      - kubectl
    state: present
```

6. **Activation de kubelet**

```
- name: Enable kubelet
  service:
    name: kubelet
    enabled: yes
```

3.5 Test de Connectivité avec le Module Ping

Avant d'exécuter le playbook, la connectivité SSH est vérifiée avec le module `ping` d'Ansible :

```
$ ansible all -i inventory.ini -m ping
```

Le résultat attendu :
— Statut `SUCCESS` et valeur `"ping": "pong"`.
— Paramètre `"changed": false` indiquant qu'aucune modification n'a été effectuée.

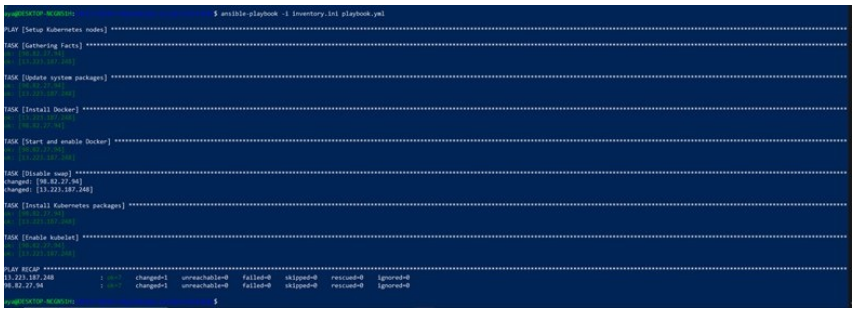
3.6 Exécution du Playbook

Le playbook est exécuté sur tous les nœuds via la commande :

```
$ ansible-playbook -i inventory.ini playbook.yml
```

3.6.1 Phases de l'exécution

- **PLAY [Setup Kubernetes nodes]** : initialisation du play.
- **TASK [Gathering Facts]** : collecte d'informations système.
- **TASK [Update system packages]** : mise à jour des paquets.
- **TASK [Install Docker]** : installation de Docker.
- **TASK [Start and enable Docker]** : démarrage du service Docker.
- **TASK [Disable swap]** : désactivation du swap.
- **TASK [Install Kubernetes packages]** : installation de kubelet, kubeadm et kubectl.
- **TASK [Enable kubelet]** : activation du service kubelet.



3.6.2 Récapitulatif final

Le récapitulatif (`PLAY RECAP`) confirme :

- `failed=0`, `skipped=0`.
- `changed` indique les tâches ayant modifié l'état des machines.
- `ok` indique les tâches déjà à l'état souhaité.

3.7 Récapitulatif des Tâches Ansible

Tâche	Description
Update system packages	Mise à jour de tous les paquets système via yum
Install Docker	Installation du runtime de conteneurs Docker
Start and enable Docker	Démarrage du service Docker et activation au démarrage
Disable swap	Désactivation du swap (prérequis Kubernetes)
Install Kubernetes packages	Installation de kubelet, kubeadm et kubectl
Enable kubelet	Activation de l'agent Kubernetes au démarrage système

3.8 Conclusion

Ce chapitre a présenté la configuration automatisée des nœuds du cluster Kubernetes à l'aide d'Ansible. Grâce à un inventaire structuré et un playbook YAML clair, Docker et les outils Kubernetes ont été installés sur les instances EC2 en une seule commande, de manière entièrement reproductible et sans intervention manuelle.

Le test de connectivité a confirmé la communication SSH opérationnelle, et l'exécution du playbook s'est déroulée sans erreur. Le chapitre suivant portera sur le développement de l'application Flask et sa conteneurisation avec Docker.

Chapitre 4

Développement et Conteneurisation de l'Application

4.1 Présentation de l'application

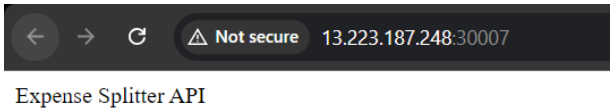
Dans le cadre de ce projet DevOps, nous avons développé un microservice basé sur le framework **Flask** (*Python*). Ce choix se justifie par la légèreté du framework et sa facilité d'intégration dans des environnements conteneurisés.

L'application remplit les fonctions suivantes :

— **Serveur Web** : Exposition d'une interface utilisateur dynamique.

— **Introspection Système** : Affichage des métadonnées de l'environnement d'exécution (hostname du Pod, IP interne).

— **Health Check** : Mise à disposition d'un endpoint `/health` permettant à Kubernetes de surveiller l'état de santé du conteneur.



4.2 Conteneurisation avec Docker

La conteneurisation permet d'isoler l'application et ses dépendances dans une unité logique standardisée. Contrairement à une machine virtuelle, le conteneur partage le noyau de l'hôte, ce qui optimise la consommation des ressources.

app.py | playbook.yml | Dockerfile

app > Dockerfile

```
1
2   FROM python:3.11-slim
3
4
5   WORKDIR /app
6
7
8   COPY requirements.txt .
9
10
11  RUN pip install --no-cache-dir -r requirements.txt
12
13
14  COPY . .
15
16  EXPOSE 5000
17
18
19  CMD ["python", "app.py"]
```

4.2.1 Structure du Dockerfile

Le fichier `Dockerfile` définit les étapes de construction de l'image de manière déclarative. Il inclut typiquement :

- Base image (Python)
- Copie du code source
- Installation des dépendances
- Définition du point d'entrée (`ENTRYPOINT`/`CMD`)

4.2.2 Construction de l'image

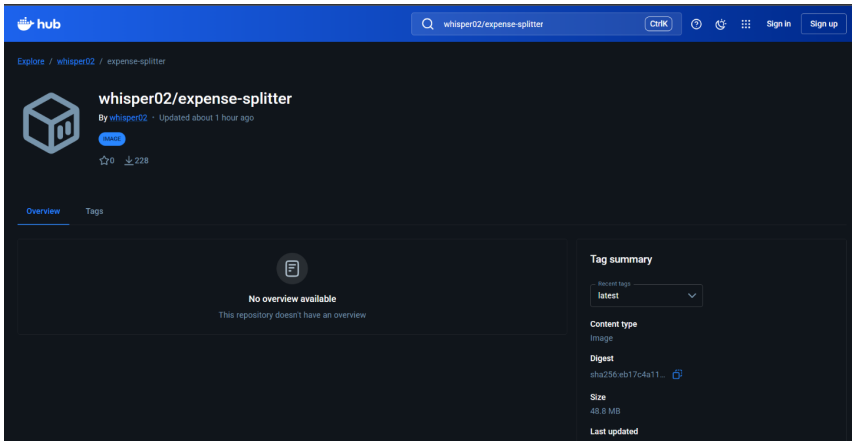
L'image est générée avec la commande suivante :

```
docker build -t <username>/flask-app:1.0 .
```

Cette commande crée une image immuable prête à être déployée sur le cluster Kubernetes.

4.3 Publication de l'image sur Docker Hub

Pour permettre aux nœuds du cluster Kubernetes d'accéder à l'image, celle-ci doit être stockée dans un registre centralisé. Dans ce projet, nous avons utilisé **Docker Hub** comme registre public.



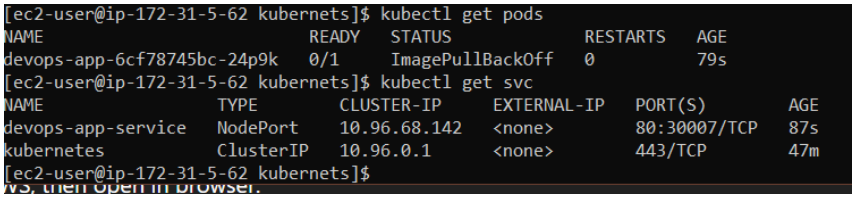
Les étapes sont les suivantes :

1. **Tagging** : Attribution d'une version spécifique pour assurer la traçabilité.
2. **Push** : Transfert de l'image vers le dépôt distant via la commande :

   ```
   docker push <username>/flask-app:1.0
   ```

4.4 Déploiement sur Kubernetes (Orchestration)

Une fois l'image disponible sur le registre, nous procédons à l'orchestration via Kubernetes pour garantir la haute disponibilité et la scalabilité.



```
[ec2-user@ip-172-31-5-62 kubernets]$ kubectl get pods
NAME                          READY   STATUS             RESTARTS   AGE
devops-app-6cf78745bc-24p9k   0/1     ImagePullBackOff   0          79s
[ec2-user@ip-172-31-5-62 kubernets]$ kubectl get svc
NAME                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
devops-app-service   NodePort    10.96.68.142   <none>        80:30007/TCP   87s
kubernetes           ClusterIP   10.96.0.1      <none>        443/TCP        47m
[ec2-user@ip-172-31-5-62 kubernets]$
```

4.4.1 Deployment (deployment.yaml)

Le `Deployment` définit l'état souhaité de l'application. Dans notre projet :

— **Nombre de réplicas** : 2, assurant la redondance.

— En cas de défaillance d'un Pod, le Control Plane Kubernetes crée automatiquement un nouveau Pod pour maintenir l'état souhaité.

4.4.2 Service (service.yaml)

Le `Service` définit l'accès réseau à l'application :

— Type : `NodePort` ou `LoadBalancer`.

— Fonction : expose l'application sur un port spécifique des instances EC2, permettant un accès externe via navigateur ou API.

4.4.3 Validation du déploiement

L'application des manifestes Kubernetes s'effectue avec la commande :

```
kubectl apply -f k8s/
```

Cette commande crée les ressources définies dans les fichiers YAML et déploie l'application sur le cluster.

Chapitre 5

Pipeline CI/CD avec GitHub Actions

5.1 Introduction au CI/CD

L'intégration continue (**CI**) et le déploiement continu (**CD**) visent à automatiser le cycle de vie du développement logiciel. L'objectif principal est de réduire le *Time-to-Market* et d'éliminer les erreurs humaines liées aux déploiements manuels.

Dans ce projet, nous avons choisi **GitHub Actions** pour sa flexibilité et son intégration native avec notre gestionnaire de version `GitHub`.

5.2 Structure du pipeline

Le workflow est défini dans le fichier `.github/workflows/deploy.yml`. Il est déclenché par un événement `push` sur la branche principale (`main`), garantissant que toute modification validée est immédiatement mise en production.

5.3 Étapes du pipeline (Workflow Jobs)

Le pipeline CI/CD se décompose en plusieurs étapes automatisées :

1. **Checkout** : Récupération du code source depuis le dépôt GitHub sur le runner.
2. **Build & Push** : Construction de l'image Docker avec un tag dynamique et transfert vers Docker Hub.
3. **SSH Deployment** : Connexion sécurisée à l'instance Master EC2 via SSH.
4. **Kubernetes Rollout** : Mise à jour de l'image du déploiement avec la commande :

   ```
   kubectl set image deployment/flask-app flask-app=<username>/flask-app:1.0
   ```

   Kubernetes effectue alors un *Rolling Update*, remplaçant les anciens Pods par les nouveaux sans interruption de service.

5.4 Sécurité et gestion des Secrets

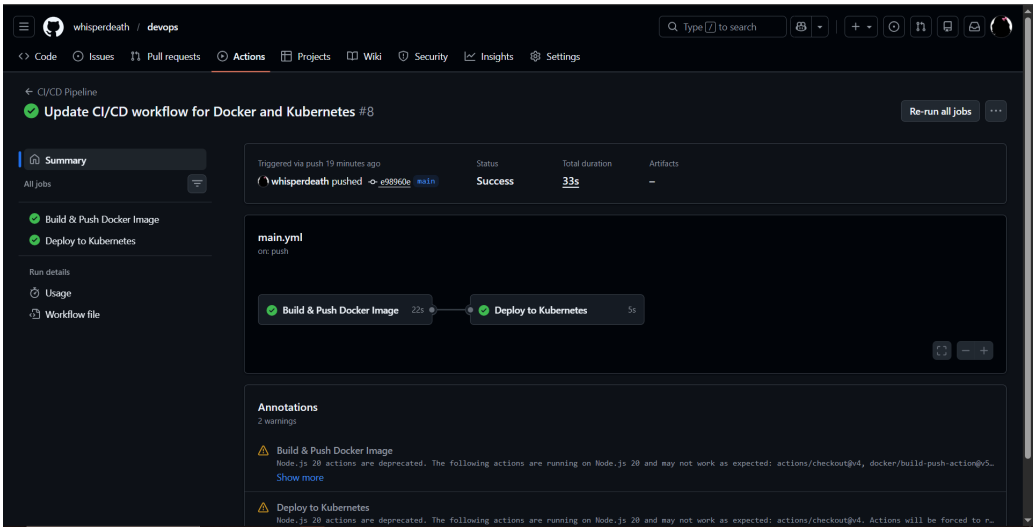
La sécurité est un aspect critique du pipeline CI/CD. Les informations sensibles, telles que :

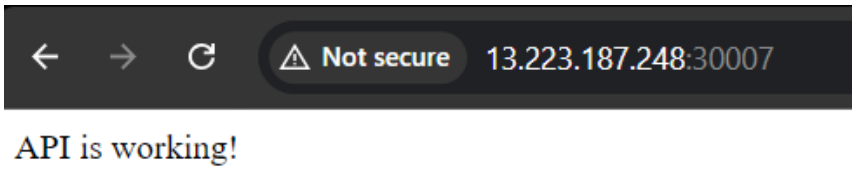
— Identifiants Docker
— Clés privées SSH
— Adresses IP des instances

ne sont jamais stockées en clair dans le code. Elles sont configurées dans les **GitHub Secrets** et injectées sous forme de variables d'environnement lors de l'exécution du workflow.

5.5 Résultat : Pipeline en action

Chaque étape du workflow est monitorée en temps réel depuis l'interface GitHub Actions. La validation visuelle du pipeline confirme le succès de l'automatisation : toutes les étapes sont exécutées sans erreur, de la construction de l'image au déploiement final sur Kubernetes.





5.6 Test de validation End-to-End

Pour valider l'intégralité de la chaîne CI/CD, nous avons effectué un test complet :

— Une modification du code source a été poussée sur la branche principale.
— Le pipeline s'est déclenché automatiquement.
— L'image Docker a été mise à jour et publiée sur Docker Hub.
— Les Pods Kubernetes ont été redéployés sur AWS EC2 en moins de deux minutes.

Ce test confirme l'efficacité de notre architecture DevOps et la fiabilité du pipeline CI/CD automatisé.

Conclusion

Le projet présenté dans ce rapport illustre la mise en place complète d'une architecture DevOps, depuis le développement d'une application jusqu'à son déploiement automatisé sur le cloud. Grâce à l'intégration de plusieurs outils modernes — Terraform pour le provisionnement de l'infrastructure, Ansible pour la configuration des machines, Docker pour la conteneurisation, Kubernetes pour l'orchestration et GitHub Actions pour l'automatisation du pipeline CI/CD — nous avons construit une chaîne DevOps cohérente et entièrement automatisée.

Les principaux bénéfices observés au cours de ce projet sont les suivants :

- **Automatisation et reproductibilité** : toutes les étapes, de la création des instances EC2 à la mise à jour des applications, sont réalisées sans intervention manuelle.
- **Fiabilité et scalabilité** : Kubernetes assure la haute disponibilité des services et la gestion automatique de la scalabilité en cas de charge accrue.
- **Réduction des erreurs humaines** : le pipeline CI/CD garantit que chaque modification du code est testée, construite et déployée de manière standardisée.
- **Gain de temps et efficacité opérationnelle** : la combinaison d'IaC et de l'automatisation des déploiements réduit considérablement le temps nécessaire pour passer du développement à la production.

En conclusion, ce projet démontre l'importance et l'efficacité d'une approche DevOps complète pour la gestion moderne des applications et de l'infrastructure cloud. Il constitue une base solide pour l'implémentation de solutions professionnelles évolutives et fiables dans un environnement technologique en constante évolution.